2026-8-20 Linux线程--生产者消费者模型

- 1. 生产者消费者模型
  - 1.1 为什么需要生产者消费者模型
  - 1.2 三个关系、两个角色和一个交易场所
  - 1.3 互斥与同步各自负责什么
- 2. 用 mutex 和条件变量实现阻塞队列
  - 2.1 阻塞队列的设计
  - 2.2 第一版代码
  - 2.3 第一版运行现象
  - 2.4 第一版存在的问题
- 3. 优化后的阻塞队列
  - 3.1 用 while 重新检查条件
  - 3.2 记录等待线程数量
  - 3.3 从单生产单消费扩展到多生产多消费
- 4. 用 POSIX 信号量实现环形缓冲区
  - 4.1 把缓冲区容量看成资源数量
  - 4.2 生产者和消费者分别关心什么资源
  - 4.3 环形队列的队头与队尾
  - 4.4 信号量版生产者消费者代码
  - 4.5 为什么仍然需要互斥量
  - 4.6 非空非满时允许同时推进
- 5. 两种实现方式的比较
- 6. 生产者消费者模型的核心结论

1. 生产者消费者模型

生产者消费者模型通过一个共享缓冲区连接两类线程：

- 生产者负责生成数据或任务，并写入缓冲区。
- 消费者负责从缓冲区取出数据或任务，再独立处理。
- 共享缓冲区通常由阻塞队列或环形队列实现。

```
生产者线程   --->   共享缓冲区   --->   消费者线程
   生产                缓存/同步              消费
```

生产者不需要等待消费者立刻处理，消费者也不需要直接向生产者索取。缓冲区把两类线程之间的强依赖转化为对共享数据结构的同步访问。

1.1 为什么需要生产者消费者模型

如果生产者和消费者直接传递数据，双方速度必须紧密匹配：生产者处理慢时，消费者只能等待；消费者处理慢时，生产者又可能被阻塞。共享缓冲区可以把二者解耦：

- **解耦**：生产者只负责生产，消费者只负责消费，双方不必知道对方具体是谁。
- **支持并发**：生产者生成数据和消费者处理数据可以同时进行。
- **平衡忙闲不均**：短时间的速度差由缓冲区吸收。
- **支持任务调度**：缓冲区中不仅可以放普通数据，也可以放任务对象、仿函数或函数对象。

共享缓冲区有容量上限。队列为空时，消费者应等待数据；队列已满时，生产者应等待空间。这两个方向共同构成了生产者消费者模型的核心同步关系。

1.2 三个关系、两个角色和一个交易场所

生产者消费者模型可以用“321 原则”记忆。

三个关系：

线程关系	需要解决的问题
生产者与生产者	不能同时修改队尾或同一块空闲区域
消费者与消费者	不能同时取走同一个数据，也不能同时修改队头
生产者与消费者	既要互斥访问缓冲区，又要同步等待数据或空位

两个角色：生产者和消费者。

一个交易场所：共享缓冲区，例如 `std::queue` 或有界环形队列。

1.3 互斥与同步各自负责什么

- **互斥**保证同一时刻只有一个线程修改队列内部状态。
- **同步**保证队列为空时消费者等待，队列满时生产者等待，并在条件满足后按合理顺序继续。

生产者虽然不能和另一个线程同时修改队列结构，但消费者取出任务后，可以在队列之外处理任务。此时生产者可以继续向队列写入新任务，因此“获取任务”和“处理任务”仍然可以并发。

临界区只应覆盖队列操作，不应把耗时的业务处理放进锁内。否则消费者即使已经取到任务，也会长期占用队列锁，降低生产者和消费者的整体并发度。

2. 用 mutex 和条件变量实现阻塞队列

2.1 阻塞队列的设计

一个线程安全的有界阻塞队列需要保存以下状态：

```
std::queue<T> queue_;           // 共享数据
std::size_t capacity_;          // 容量
pthread_mutex_t mutex_;         // 保护队列状态
pthread_cond_t not_empty_;      // 消费者等待队列非空
pthread_cond_t not_full_;       // 生产者等待队列未满
int waiting_consumers_ = 0;     // 等待数据的消费者数量
int waiting_producers_ = 0;     // 等待空间的生产者数量
```

互斥锁保护队列，两个条件变量分别表达两个方向的等待原因：

- 消费者等待 `not_empty_`，因为队列为空时无法取出数据。
- 生产者等待 `not_full_`，因为队列已满时无法写入数据。

队列状态判断本身就是临界区的一部分。锁必须覆盖“判断条件、修改队列、检查对端是否在等待、发出通知”这一完整过程。

2.2 第一版代码

第一版使用 `if` 检查一次等待条件：

```
template <typename T>
class BlockQueue {
public:
    explicit BlockQueue(int capacity = 5)
        : capacity_(capacity)
    {
        pthread_mutex_init(&mutex_, nullptr);
```

```
        pthread_cond_init(&consumer_cond_, nullptr);
        pthread_cond_init(&producer_cond_, nullptr);
    }

    void Enqueue(T& value)
    {
        pthread_mutex_lock(&mutex_);

        if (queue_.size() == capacity_) {
            pthread_cond_wait(&producer_cond_, &mutex_);
        }

        queue_.push(value);
        pthread_cond_signal(&consumer_cond_);
        pthread_mutex_unlock(&mutex_);
    }

    void Pop(T* out)
    {
        pthread_mutex_lock(&mutex_);

        if (queue_.empty()) {
            pthread_cond_wait(&consumer_cond_, &mutex_);
        }

        *out = queue_.front();
        queue_.pop();
        pthread_cond_signal(&producer_cond_);
        pthread_mutex_unlock(&mutex_);
    }

private:
    std::queue<T> queue_;
    int capacity_;
    pthread_mutex_t mutex_;
    pthread_cond_t consumer_cond_;
    pthread_cond_t producer_cond_;
};
```

线程在等待之前必须持有互斥锁。`pthread_cond_wait` 会自动释放锁并阻塞；被唤醒后，它会重新竞争并取得锁，然后才返回。这样生产者才可能在消费者等待期间进入临界区写入数据。

2.3 第一版运行现象

当消费者执行 `sleep(5)` 时，生产者可以先连续生产，直到队列达到容量上限：

```
ZL@VM-0-14-ubuntu:~/code/lesson32/1.BlockQueue$ ./cp_bq
生产10
生产11
生产12
生产13
生产14
消费10
生产15
消费11
生产16
消费12
生产17
消费13
生产18
消费14
生产19
```

这个输出体现了有界缓冲区的基本行为：生产者不会无限领先，队列满后必须等待消费者取走数据。消费者每次取走一个元素并通知生产者后，生产者才能继续放入下一个元素。

如果双方速度接近，或者运行的初始时序不同，也可能看到更接近“生产一个、消费一个”的交替顺序：

```
ZL@VM-0-14-ubuntu:~/code/lesson32/1.BlockQueue$ ./cp_bq
生产10
消费10
生产11
消费11
生产12
消费12
生产13
消费13
生产14
消费14
生产15
消费15
生产16
消费16
生产17
消费17
```

这里的交替不是强制规定，而是两个条件变量根据队列空满状态形成的合理等待关系。队列始终在队尾生产、队头消费，并保持先进先出。

2.4 第一版存在的问题

第一版用 `if` 只检查一次条件，但线程从 `pthread_cond_wait` 返回后，条件不一定仍然成立：

- 条件变量允许伪唤醒。
- 一个通知可能对应多个等待线程。
- 多个消费者被唤醒后，第一个消费者可能已经取走唯一的数据。
- 多个生产者被唤醒后，第一个生产者可能已经填满最后一个空位。

消费者可能因此继续读取空队列，生产者也可能继续写入已满队列。错误是否出现取决于调度时序，因此程序看起来有时正确，实际上并不可靠。

另外，无条件调用 `pthread_cond_signal` 还可能唤醒根本没有等待者的条件变量，产生无效通知。

3. 优化后的阻塞队列

3.1 用 while 重新检查条件

条件变量只负责通知线程“可以重新检查”，不保证条件已经成立。因此等待必须放进 `while` 循环：

```
while (condition_is_false) {
    pthread_cond_wait(&cond, &mutex);
}
```

消费者醒来后再次检查队列是否为空；生产者醒来后再次检查队列是否已满。只有条件真正成立，线程才退出循环并操作队列。

3.2 记录等待线程数量

为了减少无效唤醒，可以分别记录生产者和消费者的等待数量。计数在持有互斥锁时修改，因此也是受保护的共享状态。

```
void Enqueue(const T& value)
{
    pthread_mutex_lock(&mutex_);

    while (queue_.size() == capacity_) {
```

```
        ++waiting_producers_;
        pthread_cond_wait(&not_full_, &mutex_);
        --waiting_producers_;
    }

    queue_.push(value);

    if (waiting_consumers_ > 0) {
        pthread_cond_signal(&not_empty_);
    }

    pthread_mutex_unlock(&mutex_);
}

T Pop()
{
    pthread_mutex_lock(&mutex_);

    while (queue_.empty()) {
        ++waiting_consumers_;
        pthread_cond_wait(&not_empty_, &mutex_);
        --waiting_consumers_;
    }

    T value = queue_.front();
    queue_.pop();

    if (waiting_producers_ > 0) {
        pthread_cond_signal(&not_full_);
    }

    pthread_mutex_unlock(&mutex_);
    return value;
}
```

完整成员和生命周期管理如下：

```
template <typename T>
class BlockQueue {
public:
    explicit BlockQueue(std::size_t capacity = 5)
        : capacity_(capacity)
    {
        pthread_mutex_init(&mutex_, nullptr);
        pthread_cond_init(&not_empty_, nullptr);
```

```
        pthread_cond_init(&not_full_, nullptr);
    }

    BlockQueue(const BlockQueue&) = delete;
    BlockQueue& operator=(const BlockQueue&) = delete;

    // Enqueue 和 Pop 使用上面的优化版本

    ~BlockQueue()
    {
        pthread_mutex_destroy(&mutex_);
        pthread_cond_destroy(&not_empty_);
        pthread_cond_destroy(&not_full_);
    }

private:
    std::queue<T> queue_;
    std::size_t capacity_;
    pthread_mutex_t mutex_;
    pthread_cond_t not_empty_;
    pthread_cond_t not_full_;
    int waiting_consumers_ = 0;
    int waiting_producers_ = 0;
};
```

3.3 从单生产单消费扩展到多生产多消费

优化后的阻塞队列天然可以扩展到多生产、多消费：

- 所有生产者共享 `mutex_`，因此不会同时破坏队尾。
- 所有消费者共享 `mutex_`，因此一个元素只会被一个消费者取走。
- 队列中的元素可以统一调度，但每个消费者应该只处理自己成功取出的任务。

主程序只需增加生产者或消费者线程，并把同一个队列地址传给它们：

```
BlockQueue<int> queue(5);

pthread_t producer1;
pthread_t producer2;
pthread_t consumer1;
pthread_t consumer2;

pthread_create(&producer1, nullptr, ProducerRoutine, &queue);
```

```
pthread_create(&producer2, nullptr, ProducerRoutine, &queue);
pthread_create(&consumer1, nullptr, ConsumerRoutine, &queue);
pthread_create(&consumer2, nullptr, ConsumerRoutine, &queue);
```

队列模板不仅可以存整数，也可以存任务对象。生产者把待执行任务放入队列，消费者取出任务后，在队列之外执行，这为后续的日志系统、任务队列和线程池提供了基础。

4. 用 POSIX 信号量实现环形缓冲区

4.1 把缓冲区容量看成资源数量

除了 `std::queue + mutex + 条件变量`，生产者消费者模型也可以用信号量实现。信号量的本质是一个资源计数器：

- `P(sem)` 申请一个资源。计数大于 `0` 时减一并继续；计数为 `0` 时阻塞。
- `V(sem)` 归还一个资源，将计数加一，并唤醒可能的等待者。

计数为 `1` 的二元信号量可以实现互斥；计数大于 `1` 的信号量可以表示多份同类资源。

4.2 生产者和消费者分别关心什么资源

对于容量为 `N` 的缓冲区：

```
sem_t blank_sem; // 初始值为 N，表示空位数量
sem_t data_sem;  // 初始值为 0，表示已有数据数量
```

- 生产者需要先取得一个空位，所以执行 `P(blank_sem)`。
- 消费者需要先取得一份数据，所以执行 `P(data_sem)`。
- 生产者写入数据后执行 `V(data_sem)`，告诉消费者“多了一份数据”。
- 消费者取走数据后执行 `V(blank_sem)`，告诉生产者“多了一个空位”。

信号量的申请和释放是原子操作，因此可以用计数准确地表示空位与数据数量。

4.3 环形队列的队头与队尾

环形队列用一个固定长度数组保存数据：

```
T ring[N];
std::size_t head = 0; // 消费者从这里取数据
std::size_t tail = 0; // 生产者从这里写数据
```

每次写入或读取后，下标循环前进：

```
tail = (tail + 1) % N;
head = (head + 1) % N;
```

环形队列需要遵守四条规则：

1. 队列为空时，`head == tail`，消费者不能取数据。
2. 队列已满时，`head == tail`，生产者不能写入数据。
3. 生产者写入的位置不能追上消费者的位置。
4. 消费者读取的位置不能超过生产者的位置。

空和满时 `head == tail` 的形式相同，因此不能只靠两个下标区分空满，必须额外使用计数器或信号量表示“当前有多少个空位、多少个数据”。

4.4 信号量版生产者消费者代码

下面的代码使用 `blank_sem` 和 `data_sem` 控制同步，再用一把互斥量保护 `head`、`tail` 以及数组槽位的访问：

```
#include <cstddef>
#include <semaphore.h>
#include <pthread.h>

template <typename T, std::size_t N>
class RingBuffer {
public:
    RingBuffer()
    {
        sem_init(&blank_sem_, 0, N);
        sem_init(&data_sem_, 0, 0);
        pthread_mutex_init(&mutex_, nullptr);
    }

    RingBuffer(const RingBuffer&) = delete;
```

```
    RingBuffer& operator=(const RingBuffer&) = delete;

    void Push(const T& value)
    {
        // 1. 等待一个空位
        sem_wait(&blank_sem_);

        // 2. 互斥地修改队尾和槽位
        pthread_mutex_lock(&mutex_);
        ring_[tail_] = value;
        tail_ = (tail_ + 1) % N;
        pthread_mutex_unlock(&mutex_);

        // 3. 新增一份可消费的数据
        sem_post(&data_sem_);
    }

    T Pop()
    {
        // 1. 等待一份数据
        sem_wait(&data_sem_);

        // 2. 互斥地修改队头和槽位
        pthread_mutex_lock(&mutex_);
        T value = ring_[head_];
        head_ = (head_ + 1) % N;
        pthread_mutex_unlock(&mutex_);

        // 3. 归还一个空位
        sem_post(&blank_sem_);
        return value;
    }

    ~RingBuffer()
    {
        pthread_mutex_destroy(&mutex_);
        sem_destroy(&blank_sem_);
        sem_destroy(&data_sem_);
    }

private:
    T ring_[N];
    std::size_t head_ = 0;
    std::size_t tail_ = 0;
    sem_t blank_sem_;
    sem_t data_sem_;
```

```
    pthread_mutex_t mutex_;
};
```

`sem_wait` 对应 `P` 操作，`sem_post` 对应 `V` 操作。

4.5 为什么仍然需要互斥量

空位信号量和数据信号量解决的是“有没有资源”的同步问题，但多个生产者仍可能同时修改 `tail_`，多个消费者也可能同时修改 `head_`。

因此正确的顺序是：

1. 生产者先用 `sem_wait(&blank_sem_)` 等待空位。
2. 再用 `mutex_` 保护 `ring_[tail_]` 和 `tail_` 的修改。
3. 解锁后执行 `sem_post(&data_sem_)` 通知消费者。

消费者使用相反的顺序取得数据和归还空位。等待资源时不要占着互斥锁，否则会降低并发度，严重时还可能形成循环等待。

4.6 非空非满时允许同时推进

当 `head != tail` 且缓冲区既不为空也不为满时：

- 生产者访问 `tail` 指向的空位。
- 消费者访问 `head` 指向的数据。

二者操作的是不同位置，但修改队头、队尾仍受同一把互斥量保护。信号量确保资源数量约束，互斥量确保队列元数据不会同时被错误修改。

5. 两种实现方式的比较

实现方式	同步机制	主要特点
`std::queue + mutex + 条件变量`	条件变量等待“非空/未满”	直接表达队列状态，适合动态容器
环形数组 + mutex + POSIX 信号量	`blank/data` 信号量计数	固定容量、无动态分配，生产消费位置清晰

两种实现都必须同时解决：

- 生产者与生产者之间的互斥。
- 消费者与消费者之间的互斥。
- 生产者与消费者之间对空位、数据的同步。

6. 生产者消费者模型的核心结论

- 生产者消费者模型通过共享缓冲区解耦两类线程，缓冲区的容量必须有限。
- 队列满时生产者等待空位，队列空时消费者等待数据。
- 生产者之间、消费者之间需要互斥；生产者与消费者之间既需要互斥，也需要同步。
- 条件变量必须配合互斥锁和 `while` 条件检查使用，不能依赖一次 `if` 或一次唤醒。
- POSIX 信号量可以用空位和数据数量描述生产消费关系，但多个线程修改队头、队尾时仍需要互斥锁。
- 临界区应只覆盖队列操作，消费者取得任务后的耗时处理应放到锁外，才能真正提高并发度。